

说出并解释下列代码的输出结果：

```
function Foo() {  
  getName = function () {  
    console.log(1);  
  };  
  return this;  
}  
Foo.getName = function () {  
  console.log(2);  
};  
Foo.prototype.getName = function () {  
  console.log(3);  
};  
var getName = function () {  
  console.log(4);  
};  
function getName() {  
  console.log(5);  
}  
Foo.getName();  
Foo().getName();  
getName();  
new Foo.getName();  
new Foo().getName();  
new new Foo().getName();
```

参考答案

1、 Foo.getName();

调用Foo的静态方法，所以，打印2

2、 Foo().getName();

Foo()就是普通函数调用，返回的this是window，后面调用window.getName()而window下的getName在Foo()中调用getName被重新赋值,所以,打印1

3、 getName();

在执行过Foo().getName()的基础上，所以getName=function(){console.log(1)},所以,打印1，[如果getName()放在Foo().getName()上执行打印结果为4]

4、 new Foo.getName();

构造器私有属性的getName(),所以,打印2

5、 new Foo().getName();

原型上的getName(), 打印3

6、 new new Foo().getName()

首先new Foo()得到一个空对象 {}

第二步向空对象中添加一个属性getName, 值为一个函数

第三步new {}.getName()

等价于 var bar = new (new Foo().getName()); console.log(bar)

先new Foo得到的实例对象上的getName方法, 再将这个原型上getName方法当做构造函数继续new, 所以执行原型上的方法,打印3